

□ Milestone 3 —Optimisation des filtres et rapports

Sprint V3 ·Al Qalam Stock Manager Thème : ·compréhensions ·
générateurs ·tableau de bord ·filtres temps réel

Objectif

Remplacer toutes les boucles for + append par des **compréhensions Python** (list, dict, set) et des **générateurs** (yield), et ajouter un onglet "Tableau de bord" avec KPIs, statistiques par catégorie, top 5 produits, et une recherche temps réel.

Ce qui change par rapport à V2

```
alqalamv3/  
├── services/  
│   └── stock_service.py ← + par_categorie() top_valeur() rechercher()  
│                           stats_categories() flux_export()  
├── ui/frames/  
│   ├── stock_frame.py ← + barre recherche + filtre catégorie + compteur  
│   └── rapport_frame.py ← NOUVEAU : KPIs + stats par catégorie + top 5
```

Concepts mis en œuvre

Expression	Syntaxe	Remplace
List comprehension	[x for x in iterable if condition]	for + append()
Dict comprehension	{k: v for k, v in ...}	for + dict[k] = v
Set comprehension	{expr for x in iterable}	for + set.add()
Générateur	(expr for x in ...) ou yield	Liste intermédiaire en mémoire

Guide de réalisation étape par étape

Étape 1 —List comprehension : filtres (10 min)

```
# services/stock_service.py — V1/V2

# ☐ Avant (boucle explicite)
def produits_en_alerte(self) -> list:
    alertes = []
    for p in self._produits.values():
        if p.est_en_alerte():
            alertes.append(p)
    return alertes

# ☐ Après (List comprehension)
def produits_en_alerte(self) -> list:
    return [p for p in self._produits.values() if p.est_en_alerte()]

# Filtre multi-champs : ref, nom OU catégorie contiennent le texte
def rechercher(self, texte: str) -> list:
    """
    [V3] List comprehension avec condition composée.
    Un seul parcours, pas de liste intermédiaire.
    """
    t = texte.lower().strip()
    if not t:
        return list(self._produits.values()) # tout retourner si vide
    return [
        p for p in self._produits.values()
        if t in p.ref.lower()
        or t in p.nom.lower()
        or t in p.categorie.lower()
    ]
```

Étape 2 —Dict et Set comprehension : groupement (15 min)

```
def par_categorie(self) -> dict:
    """
    Regroupe les produits par catégorie.
    [V3] Deux comprehensions imbriquées en deux lignes.
    """
    # Set comprehension : toutes les catégories uniques (sans doublon)
    categories = {p.categorie for p in self._produits.values()}

    # Dict comprehension : { catégorie -> [liste de produits] }
```

```

    return {
        cat: [p for p in self._produits.values() if p.categorie == cat]
        for cat in sorted(categories)
    }

# Démonstration
stock = StockService()
par_cat = stock.par_categorie()

for categorie, produits in par_cat.items():
    print(f"\n{categorie} ({len(produits)} produits) :")
    for p in produits:
        print(f"  {p}")

# Sortie :
# Coupe (1 produits) :
# [CIS-001] Ciseaux 17cm — Qté: 25 / Seuil: 5 ☑ OK
# Écriture (4 produits) :
# [CRAY-001] Crayon HB — Qté: 150 ...
# ...

```

Étape 3 — Dict comprehension avec générateur : statistiques (15 min)

```

def stats_categories(self) -> dict:
    """
    Statistiques agrégées par catégorie.
    [V3] Dict comprehension imbriqué + expression génératrice dans sum().

    Retourne : { cat → { nb_produits, valeur_totale, nb_alertes } }
    """
    return {
        cat: {
            "nb_produits" : len(prods),
            # Générateur dans sum() : pas de liste intermédiaire
            "valeur_totale": sum(p.valeur_stock() for p in prods),
            "nb_alertes" : sum(1 for p in prods if p.est_en_alerte()),
        }
        for cat, prods in self.par_categorie().items()
    }

```

Différence list vs générateur dans sum() :

```

produits = [p1, p2, p3, ...] # 10 000 produits

# ☑ List comprehension : crée une liste de 10 000 floats EN MÉMOIRE
total = sum([p.valeur_stock() for p in produits])

```

```
# Expression génératrice : calcule valeur par valeur, sans liste
total = sum(p.valeur_stock() for p in produits)
# ↑ pas de crochets → générateur, pas une liste
```

Étape 4 —Générateur avec yield : export CSV (10 min)

```
def flux_export(self):
    """
    Générateur de lignes CSV.
    [V3] yield : produit les données une par une sans tout charger en mémoire.

    Usage : for ligne in stock.flux_export(): csv_writer.writerow(ligne)
    """
    # Ligne d'en-tête
    yield ["ref", "nom", "categorie", "prix_achat", "prix_vente", "qte", "seuil_min"]

    # Une ligne par produit, dans l'ordre alphabétique (utilise __lt__ de V2)
    for p in sorted(self._produits.values()):
        yield [p.ref, p.nom, p.categorie,
              p.prix_achat, p.prix_vente, p.qte, p.seuil_min]
```

Pourquoi yield au lieu de return liste ?

```
# Sans générateur : charge TOUT en mémoire avant de commencer à écrire
def exporter(self) -> list:
    lignes = [en_tete]
    for p in self._produits.values():
        lignes.append([p.ref, p.nom, ...])
    return lignes # 10 000 produits → liste de 10 000 éléments en RAM

# Avec générateur : une ligne calculée, une ligne écrite, mémoire constante
for ligne in stock.flux_export():
    writer.writerow(ligne) # calcule et écrit une ligne à la fois
```

Étape 5 —Recherche temps réel dans l'UI (15 min)

```
# ui/frames/stock_frame.py — ajouts V3

class StockFrame(ctlk.CTkFrame):
    def _construire_filtres(self):
        # Champ de recherche texte
        self._var_recherche = ctlk.StringVar()
        self._champ_recherche = ctlk.CTkEntry()
```

```

        self, textvariable=self._var_recherche,
        placeholder_text="🔍 Rechercher (réf, nom, catégorie)..."
    )
    self._champ_recherche.pack(fill="x", padx=10, pady=5)
    # Déclenche la recherche à chaque frappe
    self._champ_recherche.bind("<KeyRelease>", lambda e: self.rafraichir())

    # Filtre catégorie (menu déroulant)
    self._var_categorie = ctk.StringVar(value="Toutes")
    self._menu_cat = ctk.CTkOptionMenu(
        self, variable=self._var_categorie,
        command=lambda _: self.rafraichir()
    )
    self._menu_cat.pack()

def rafraichir(self):
    texte = self._var_recherche.get()
    cat = self._var_categorie.get()

    # Appliquer les filtres (list comprehensions dans stock_service)
    produits = self._stock.rechercher(texte)
    if cat != "Toutes":
        produits = [p for p in produits if p.categorie == cat]

    # Mettre à jour le compteur
    self._lbl_compteur.configure(text=f"{len(produits)} produit(s) affiché(s)")

    # Remplir le tableau
    for item in self._arbre.get_children():
        self._arbre.delete(item)
    for p in produits:
        self._arbre.insert("", "end", iid=p.ref, values=(...))

```

Étape 6 —Tableau de bord (20 min)

ui/frames/rapport_frame.py — NOUVEAU V3

```

class RapportFrame(ctk.CTkFrame):
    """Onglet tableau de bord : KPIs + stats catégories + top 5."""

    def __init__(self, parent, stock_service):
        super().__init__(parent)
        self._stock = stock_service
        self._construire_kpis()

```

```

self._construire_stats_categories()
self._construire_top5()
self.rafraichir()

def _construire_kpis(self):
    """3 cartes : nb produits · valeur totale · nb alertes."""
    frame = ctk.CTkFrame(self)
    frame.pack(fill="x", padx=10, pady=10)

    self._kpi_produits = self._carte_kpi(frame, "Produits", "—")
    self._kpi_valeur = self._carte_kpi(frame, "Valeur stock", "—")
    self._kpi_alertes = self._carte_kpi(frame, "Alertes", "—")

def rafraichir(self):
    # Met à jour les KPIs avec les vraies données
    self._kpi_produits.configure(text=str(self._stock.nb_produits()))
    self._kpi_valeur.configure(
        text=f"{self._stock.valeur_totale_stock():.2f} TND"
    )
    self._kpi_alertes.configure(text=str(self._stock.nb_alertes()))

    # Stats par catégorie (dict comprehension)
    stats = self._stock.stats_categories()
    # ... affichage dans un tableau ...

    # Top 5 (sorted + slicing)
    top5 = self._stock.top_valeur(5)
    medailles = ["🥇", "🥈", "🥉", "4.", "5."]
    for i, p in enumerate(top5):
        # ... afficher medailles[i], p.nom, p.valeur_stock() ...

```

Récapitulatif des comprehensions utilisées

```

# 1. LIST — filtrer
alertes = [p for p in stock if p.est_en_alerte()]

# 2. LIST — filtrer multi-conditions
resultats = [p for p in stock
              if texte in p.ref.lower() or texte in p.nom.lower()]

# 3. SET — valeurs uniques
categories = {p.categorie for p in stock}

# 4. DICT — grouper

```

```

par_cat = {cat: [p for p in stock if p.categorie == cat]
            for cat in categories}

# 5. DICT imbriqué — agréger
stats = {cat: {"nb": len(ps), "valeur": sum(p.valeur_stock() for p in ps)}
         for cat, ps in par_cat.items()}

# 6. GENERATOR — export mémoire-efficace
for ligne in (p.to_dict() for p in sorted(stock)):
    writer.writerow(ligne)

```

Critères de validation

- ☐ Taper "cray" dans la recherche filtre instantanément le tableau sur les crayons
 - ☐ Sélectionner "Écriture" dans le menu ne montre que les produits d'écriture
 - ☐ Les deux filtres combinés fonctionnent ensemble
 - ☐ Le compteur "N produit(s) affiché(s)" se met à jour à chaque frappe
 - ☐ L'onglet "Tableau de bord" affiche les 3 KPIs corrects
 - ☐ Le tableau de stats affiche nb produits, valeur et alertes par catégorie
 - ☐ Le top 5 affiche les 5 produits de plus grande valeur avec médailles
 - ☐ Après une entrée/sortie de stock, le tableau de bord se met à jour
-

Pour aller plus loin

- Utiliser `itertools.groupby()` comme alternative aux dict comprehensions
 - Ajouter un graphique à barres des valeurs par catégorie (avec `tkinter.Canvas`)
 - Exporter le rapport complet en TXT via `flux_export()` + `open()`
-

Al Qalam Stock Manager — Formation Python Partie II — Sprint V3